M20 complete step by step run guide

This guide runs the Group 09 M20 quadruped project from a standalone assigned group folder. The current release uses a dense industrial yard, three alternating barriers and two 8 cm terrace climbs. It supplies live physics with FSM overlays, actual 500 Hz telemetry, an interactive business model and reproducible final-study statistics.

The robot is the supplied hybrid canine quadruped: four articulated legs with wheels at the feet, 12 leg motors and four hub motors. It rolls between obstacles and lifts front/rear leg pairs to climb. It is not a car replacement with cosmetic legs, and it is not claimed to walk continuously with every wheel braked.

Start with the live simulation and business dashboard. Use the experiment and document sections later when you need to reproduce or submit evidence. A replay is useful for inspection but is not the live simulation required for the showcase.

Quick route through this guide: pages 2–3 install and test; pages 4–5 live demo and business model; page 6 evidence/replay; page 7 full experiment; page 8 analysis and dossier; page 9 recording; page 10 GitHub and individual submission.

Confirmed team: Arnav Saurabh Surve, E064, SAP 70362400067; Vihan Shripad Joshi, E070, SAP 70362400086; Pratik Mangesh Gaikwad, E073, SAP 70362300032. The faculty IDs were confirmed by the user. Names identify the team, not verified personal code authorship.

Verified reference: 60 independent final-course runs, 30 per policy; 4,495,972 actual 500 Hz samples in 254 CSV parts. Proposed controller: 30 accepted missions, both climbs and no harmful contacts in each. Reactive baseline: 30 timeouts with harmful contacts. This simple baseline is substantially weaker than the integrated policy; do not generalize the effect size to expert teleoperation or hardware.

Files to keep: the entire Group_09_Arnav_Vihan_Pratik folder, including M20/meshes. Do not run from inside a ZIP preview. Do not copy only the Python scripts. The group folder is self-contained; a sibling M20 folder is no longer required.

1 Extract the project and prepare Windows

1. Extract the final submission ZIP to a normal local directory with enough free disk space. The full evidence archive is large because it retains actual measured CSVs and replay states. Avoid extracting into another project with identically named files.

2. Open the extracted Group_09_Arnav_Vihan_Pratik folder. Confirm that run_demo.ps1, requirements-lock.txt, M20/M20.xml, M20/meshes, src, analytics, tests and docs are present. The GitHub-only package omits videos and historical replay binaries; the final 500 Hz CSV evidence is retained.

3. Install CPython 3.12 for Windows if it is not already installed. The verified environment used Python 3.12.14 and MuJoCo 3.11.0. The pinned requirements are authoritative; do not upgrade packages during a recorded comparison.

4. Open PowerShell in the group folder. In File Explorer, type powershell in the address bar and press Enter. Check the directory and Python:

```
Get-Location
Get-ChildItem
py -3.12 --version
```

5. If the py command is unavailable, use the full path to a Python 3.12 executable. Do not use Python 3.13 merely because it is the first interpreter on PATH.

6. Run setup. This creates a local .venv and installs the exact lock file. Network access is normally needed the first time.

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\run_demo.ps1 -Setup
```

An explicit interpreter is also supported:

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\run_demo.ps1 -Setup -Python
'C:\path\to\Python312\python.exe'
```

Replace that example path with the real interpreter path. ExecutionPolicy Bypass applies only to this PowerShell process; no global policy change is needed. Wait for Setup complete. If pip fails, read its error, restore connectivity and rerun setup.

2 Verify the environment before demonstrating

Use the environment inside the extracted group folder. Activation is optional because every command below invokes its Python explicitly.

```
.\.venv\Scripts\python.exe --version
.\.venv\Scripts\python.exe -m pip check
.\.venv\Scripts\python.exe -m pytest -q
```

The current test suite checks model anatomy, actuator limits, mapping, ray behavior, delayed commands, acceptance gates, measured logging, statistics and business mathematics. The final verification report records the actual pass count. A test failure is a reason to investigate, not to edit a receipt manually.

Check the model directly:

```
.\.venv\Scripts\python.exe -c "import mujoco; m=mujoco.MjModel.from_xml_path('M20/M20.xml');
print(m.nq,m.nv,m.nu)"
```

Expected dimensions are 23, 22, 16. A missing STL error usually means the folder structure was broken during extraction. Restore M20/meshes and retain relative paths.

Run the default full course headlessly:

```
.\.venv\Scripts\python.exe M20\simulation.py --mode course_climb --seed 42 --out
analytics\local_runs\first_course
```

Wait for the measured summary. Seed 42 without added perturbations was verified at 121.394 simulated seconds, two climbs, zero harmful contact steps, accepted=true and goal error about 0.107744 m. Wall time depends on the computer. Compare summary fields, not an animation impression.

Exit code 0 means accepted. Exit code 2 means the measured acceptance criteria were not met; this is expected for intentionally blocked or short runs. A 15-second diagnostic does not have time to finish a two-climb course and must not be described as a failed model installation.

If the display fails but headless physics works, check graphics drivers and local desktop availability. Do not replace failed rendering with a fabricated screenshot. On low-memory systems, close unrelated programs and avoid running multiple full benchmarks together.

3 Run and explain the live robot demo

Launch the native viewer from the group folder:

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\run_demo.ps1 -Live
```

Equivalent direct command:

```
.\.venv\Scripts\python.exe src\live_m20_viewer.py --mode course_climb --seed 42 --out
analytics\local_runs\live_demo
```

The viewport is actual stepped MuJoCo physics. Its overlay reports mode/policy, simulated time, current FSM state, controller decision, harmful contact steps, verified climb count and the actual 500 Hz saved telemetry rate. The camera follows the robot.

Watch the sequence: WAKEUP raises the body; DETOUR_NAV maps and routes around alternating barriers; DOCK_ALIGN aligns before the step; paired front and rear transfer phases raise and re-support the legs; CLEAR_REAR_EDGE moves clear of the lip; the controller repeats for the second terrace and approaches the goal.

Do not call the rise of the torso alone a verified climb. The evaluator requires separate front/rear airborne and support gates, progress across the measured lip, goal arrival and no harmful contact or rollover. At the end the overlay displays measured status and acceptance. Close the window to exit after the final summary.

Closing the window early records user_closed and does not fabricate completion. Run again into a new output folder if a complete demonstration is needed. Never overwrite a submitted trial to replace an inconvenient failure.

The course is a static industrial yard with cylinders, crates, pipes, walls, three alternating tall barriers and low terraces. Mapping uses ideal rays and exact simulator pose. The reference used for scoring is generated offline for evaluation and is not sent to the controller. Unknown terrain means the controller builds its map during motion, not that the simulator lacks geometry.

Live rendering can be slower than simulated time, especially while recording or performing disk-heavy work. Rehearse on the actual presentation computer. The --fast flag removes real-time throttling; it cannot make a slow physics/rendering computer meet a fixed wall-clock speed. Do not label an accelerated or edited segment as uninterrupted real time.

4 Run the business model beside the simulation

Open a second PowerShell window in the same group folder. Leave the live robot window running and launch the interactive business model:

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\run_demo.ps1 -Business
```

Arrange the two windows side by side. The dashboard shows normalized payback, NCRI, queue delay and labor reallocation. Drag operating parity, capital parity or requests/week. Reset restores the base inputs. Explain why the curves move, rather than reading a single favorable number.

The base scenario assumes operating parity 0.22, annual baseline value 1, capital parity 0.30, five-year life, 312 reclaimed hours/year and 2080 full-time hours/year. It gives 4.615 months payback, lifetime fraction P=0.076923, NCRI=0.78 and 0.15 FTE. These are planning assumptions, not plant observations.

The queue assumes λ=8 requests/week, μ=12 services/week, one server and 40 working hours/week. Utilization is 2/3 and queue wait is 6.667 working hours. Increase requests to 12 or more: the dashboard must report no finite steady-state delay. This is the stability boundary, not a negative wait or a software failure.

Recompute machine-readable outputs and the separate telemetry-linked scenario:

```
.\.venv\Scripts\python.exe analytics\hazardous_recon_economics.py
.\.venv\Scripts\python.exe analytics\telemetry_business_bridge.py
.\.venv\Scripts\python.exe analytics\business_dashboard.py --headless
```

The bridge reads actual accepted fractions and attempt durations from the 60-run CSV. It adds assumed inspection/reset overhead of 1.5, 3 or 4.5 hours and computes hypothetical successful-job capacity. It does not convert two minutes of simulated traversal into a measured plant service rate. All assumptions and zero-success cases are retained in telemetry_business_bridge.json.

Currency-free ratios are deliberate. Increasing capital delays payback; increasing operating parity reduces NCRI; near-saturated queues grow rapidly. No NASA-TLX score, bandwidth-saving percentage or human risk reduction is inferred from robot motion.

5 Inspect telemetry and distinguish replay from live physics

Each run saves summary.json, events.json, telemetry_schema.json and numbered *_real_trials.csv parts. The scientific CSVs contain actual 2 ms samples: pose, velocities, evaluator reference, tracking error, all 16 actuator torques, FSM phase, decision, events, contact flag and support count. Parts are limited to 20,000 rows so no single raw CSV is a huge repository object.

The schema lists actuator names, units, reference polyline, sample count and each CSV SHA256. Read it before interpreting columns. The error is chassis position minus nearest point on a fixed evaluator-only path. It is not error against a time-synchronized desired motion. Integrated squared torque is effort, not electrical energy or battery consumption.

telemetry.csv and states.npz are separate 50 Hz visualization/support diagnostics. They are not the 500 Hz scientific log. states.npz permits deterministic state replay; the saved full-rate CSV is the basis for the final RMSE and effort calculations.

Replay a measured run, optionally recording a clip:

```
.\.venv\Scripts\python.exe analytics\render_yard.py analytics\local_runs\first_course --show --no-record
```

For a video, omit --no-record. Verify supported options with --help before choosing output settings. Replayed frames represent saved physical states. They do not recalculate the controller in the viewport and must be labeled replay in any evidence caption.

Inspect final-study evidence in analytics/capstone_500hz. The aggregate capstone_real_trials.csv has one row per actual trial. Each numbered trial directory retains detailed samples and summary. failure_cases.csv contains the unsuccessful baseline runs; they are part of the experiment and must remain in the submission.

The old benchmark, obstacle_course and climbing_course folders are historical studies with different protocols/source hashes. Their results are valid only for those designs. Do not mix their 50 Hz traces with the new 500 Hz study or relabel their sample sizes as current trials.

6 Reproduce the complete independent experiment

The supplied final evidence is already measured. Re-running all 60 trials is optional for ordinary demonstration, but the exact command is provided for independent reproduction. Expect meaningful compute time and several gigabytes of outputs.

```
.\.venv\Scripts\python.exe analytics\run_capstone_benchmark.py --workers 3 --out analytics\capstone_rerun
```

The script writes its design before simulations: baseline seeds 31000–31029; proposed seeds 32000–32029; 10 per policy at each roughness bound 0, 3, 6 mm. Friction, barrier positions and initial pose perturbations are seeded. The deadline per trial is 180 simulated seconds. The primary test is predeclared Welch inference on terminal XY goal error.

Use fewer workers on a lower-memory computer. Do not edit M20 source, XML or meshes during a batch; the runner compares source digests. The script refuses an existing folder without --resume. Resume accepts complete trial records with the same configuration/source and refuses ambiguous partial trials. Preserve any partial folder, then choose a new output folder if needed.

The original run finished all 60 physics trials, then exposed a NumPy Boolean serialization error while aggregating. The aggregate writer was corrected to reload the already-saved normalized JSON. All original per-trial records and CSVs were retained; no physics outcome was substituted. This repair is documented in the verification report.

Analyze the rerun and verify every raw CSV:

```
.\.venv\Scripts\python.exe analytics\analyze_capstone.py --folder analytics\capstone_rerun
```

The analyzer checks continuous 2 ms timestamps, source hashes, per-file hashes, 16 torque channels, recomputed effort and RMSE, contacts, final goal distance and both ascent gates. It writes verification.json, statistics.json and failure_cases.csv. Do not report a statistics file without checking its companion verification receipt.

Plots for the supplied study regenerate with analytics/plot_capstone.py. That script deliberately reads capstone_500hz; change its input explicitly if producing a separate rerun figure. Keep the original and rerun evidence clearly named.

7 Read the statistics and complete the dossier

The required form is docs/MDRIIA_Group_09_Technical_Dossier.docx. Its PDF is four pages. The rebuild preserves every one of the 212 original tagged Word controls, including 66 personal fields left for members. Use desktop Microsoft Word; do not flatten or replace dropdowns in Google Docs.

Technical answers use the independent final-course study. Baseline goal error is 12.3103 ± 0.0931 m; proposed is 0.1064 ± 0.0082 m. The proposed-minus-baseline difference is −12.2040 m, Welch 95% CI [−12.2388, −12.1691]. The very large pooled effect size reflects all baseline trials failing early with low variance. It is not a credible estimate of performance against an expert or a modern capable planner.

Tracking RMSE is 0.5811 ± 0.1005 m versus 0.1947 ± 0.0087 m. Total squared-torque effort is 1,128,398 ± 158,460 versus 555,625 ± 36,403 N^2m^2s. Duration and failure differ, so do not infer motor efficiency from this comparison.

Members must fill their own hours, learning, genuine authored scripts, commit share, peer reviews, sign-off and optional consent. Review Groups 10 and 03 using the actual videos; do not invent timestamps or scores. Enter the final LinkedIn post URL only after publishing the actual team recording.

After edits, export from Word using File → Export → Create PDF/XPS. Check that all required sections remain and the result has exactly four pages. Long personal answers can increase pagination; keep them concise while truthful. Every member submits the exact same signed PDF. The optional IEEE paper is separate and is not a substitute for this dossier.

The builder analytics/build_capstone_dossier.py regenerates technical answers from the original form and measured statistics. It will reset personal entries to original placeholders. Do not run it over the team's signed version; first preserve the completed personal copy. Supporting details are in M20_Extended_Technical_Report.docx and CAPSTONE_VERIFICATION_REPORT.md.

8 Record the three to four minute showcase

Use docs/SHOWCASE_RECORDING_GUIDE.md for the speaking plan. Target 3:45. Rehearse before publishing. Capture a running MuJoCo viewport with visible FSM and the business graph, plus every member's webcam and audible contribution. No slide presentation is required.

Suggested allocation: 0:00–0:25 introduction and M20 identity; 0:25–1:15 mapping and obstacle detours; 1:15–2:15 paired climbs and contact gates; 2:15–3:10 business sensitivity and queue stability; 3:10–3:45 measured comparison, AI correction, limitations and repository pointer. These are editing targets, not fabricated timestamps of a completed video.

Arnav explains mass, joints, PD control and why a climb needs lift and support gates. Vihan explains ideal rays, true pose, A*, the 500 Hz schema and comparison limits. Pratik changes a business slider, explains dimensionless payback and the queue overload boundary, and identifies which inputs are assumptions. Every student should also answer questions about the whole system.

Start screen recording before the key live actions. Show actual changing FSM text. If the laptop is slow, record a longer genuine session and edit clearly labeled live excerpts to 3–4 minutes. Do not claim an edited video is an uninterrupted real-time run. Prior MP4s are physics replays and cannot alone satisfy the live/webcam requirement.

Keep the final video out of Git. Publish natively to LinkedIn from the appropriate real account, inspect the post for playback/audio and copy its genuine URL into Section 5. The current package does not assert that posting or individual recording is complete.

Before the viva, run the two windows side by side and practice explaining a failure, a formula, a raw data row and one AI mistake. Do not memorize only favorable numbers. Bring the full offline package so network failure does not prevent the simulation, graphs, PDF or raw-evidence demonstration.

9 GitHub handoff and individual submission

The assigned destination is Group_09_Arnav_Vihan_Pratik in the course repository. The GitHub-ready ZIP contains that folder, code, models, documentation and final raw CSV evidence; it excludes MP4/NPZ and local environments. The full offline ZIP additionally retains historical evidence and replay assets. Use the correct package for the destination.

Clone the course repository or use your existing authorized checkout. Copy the prepared group folder into its assigned location and inspect changes before staging. Avoid overwriting another group's folder. Check Git identity from each student's own account; do not use an invented author name or divide artificial commits among members.

```
git status --short
git config user.name
git config user.email
git diff --stat
git add -- Group_09_Arnav_Vihan_Pratik
git diff --cached --stat
```

Each member must review, understand and make a genuine contribution, then commit from their own account. Use meaningful messages describing actual work. Follow repository branch/review rules and permissions. This package supplies no fabricated contribution history and has not been pushed on behalf of students.

Commit the final completed DOCX and the exact same signed PDF under docs/. Confirm no .venv, cache, MP4 or large generated binary is staged. The plain 500 Hz CSV chunks are intentional empirical evidence. Network upload of the full evidence set may take time; use Git rather than a browser upload limit workaround.

Open the remote group folder after pushing and verify the expected latest code and PDF are present. Record real commit hashes in the roster only after they exist. git shortlog -sn -- Group_09_Arnav_Vihan_Pratik reports actual contribution counts; it is not proof of effort by itself.

Every member must personally attach the same signed four-page PDF on Teams and click Turn in. Team-level upload does not complete individual submission. Keep a receipt/screenshot of each actual submission. The team must attend the live expo/viva and be prepared to run both the robot and business model. This guide does not claim those personal actions have happened.